

Módulo 2

Dominando CI/CD y automatización con Git

Ramas · Merge · Pull requests · GitHub Actions

Clase 2 · Duración 2 h

git — módulo 2

```
$ git checkout -b feat  
$ git commit -m "wip"  
$ git checkout main  
$ git merge feat
```

✓ 3 commits

Dónde estamos en el bucle

La clase pasada vimos el mapa completo. Hoy bajamos a dos fases concretas.



CÓDIGO

Cómo varias personas trabajan sobre el mismo código sin pisarse. Ramas, merges y revisión.

INTEGRAR

Cómo se verifica cada cambio automáticamente, sin que nadie apriete un botón. GitHub Actions.

La pregunta de hoy

Si cinco personas tocan el mismo proyecto todos los días, ¿cómo hacemos para que el resultado siga funcionando? Git responde la mitad; la automatización responde la otra mitad.

Agenda de hoy

Dos horas, cuatro bloques y un break de 10 minutos

1

Git por dentro

El modelo mental: commits, grafo, ramas como punteros

25 min

2

Ramas, merge y conflictos

Cómo se unen dos líneas de trabajo y qué pasa cuando chocan

30 min



Break

Estiren las piernas

10 min

3

Flujos de trabajo y pull requests

GitHub Flow, revisión de código, buenos commits

20 min

4

GitHub Actions

Tu primer pipeline de CI, en vivo

25 min

Al terminar la clase vas a poder...

1

Explicar qué guarda Git realmente

commits como fotos completas, no como diferencias

2

Razonar sobre el grafo de commits

y contar en qué estado queda una rama después de una secuencia de comandos

3

Crear, cambiar y fusionar ramas

entendiendo la diferencia entre fast-forward y merge commit

4

Resolver un conflicto de merge

sin entrar en pánico ni borrar la carpeta

5

Escribir un workflow de GitHub Actions

que verifique automáticamente cada push

El objetivo 2 es exactamente lo que evalúa el ejercicio de la plataforma. Le vamos a dedicar un bloque entero.

01

BLOQUE 1 · 25 MINUTOS

Git por dentro

Si entendés qué guarda Git, dejás de memorizar comandos y empezás a razonar.

El problema que Git resuelve

Todos pasamos por esto antes de aprender control de versiones

SIN CONTROL DE VERSIONES

```
proyecto/  
proyecto_v2/  
proyecto_final/  
proyecto_final_OK/  
proyecto_final_OK_ahora_si/  
proyecto_ESTES (1).zip
```

CON GIT

- Una sola carpeta, con toda la historia adentro
- Podés volver a cualquier punto del pasado
- Sabés quién cambió qué, cuándo y por qué
- Varias personas trabajan en paralelo sin pisarse

Git es distribuido

Cada persona tiene una copia completa del historial en su máquina. Podés trabajar, hacer commits y ver la historia entera sin internet. GitHub no es Git: es un servidor donde las copias se encuentran.

Las tres áreas de Git

Todo comando de Git mueve archivos entre estas tres cajas

1

Working directory

Tu carpeta de trabajo

Los archivos que ves y editás con tu editor. Git los ve como 'modificados'.

2

Staging area

La antesala del commit

Lo que decidiste incluir en el próximo commit. Acá elegís qué se guarda y qué no.

3

Repository

El historial permanente

Los commits ya guardados. Esto es lo que se sincroniza con GitHub.

● ● ● el viaje de un cambio

```
git add archivo.txt      # working directory --> staging
git commit -m "mensaje"  # staging --> repository
git push                 # repository local --> GitHub
```

Qué es realmente un commit

No es una diferencia. Es una foto completa del proyecto, más metadatos.

UN COMMIT CONTIENE

Snapshot	el estado completo de todos los archivos
Autor y fecha	quién lo hizo y cuándo
Mensaje	por qué lo hizo
Hash	un identificador único, ej. a3f1e2c
Padre	el commit anterior — así se forma la cadena

EL DETALLE QUE IMPORTA

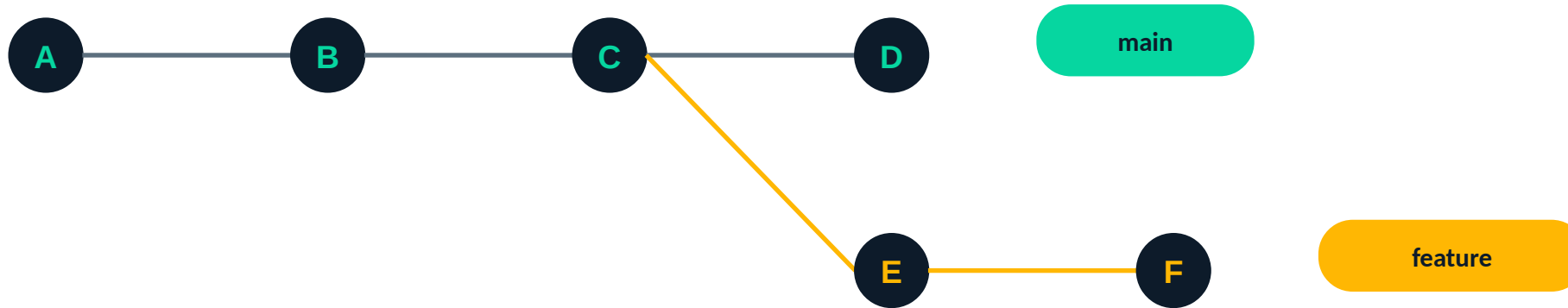
Cada commit apunta a su padre. Esa cadena de punteros ES el historial.

Por eso Git puede reconstruir cualquier momento del pasado: no guarda instrucciones para llegar ahí, guarda el estado exacto. Y por eso el hash cambia si cambia cualquier cosa del commit, incluido su padre.

Un commit no se modifica nunca. Si "cambiás" un commit, Git crea uno nuevo con otro hash.

El historial es un grafo

Cada commit apunta hacia atrás, a su padre. Las ramas hacen que el grafo se abra.



Cómo leerlo

El proyecto arrancó en A. En C alguien abrió una rama para trabajar aparte. Mientras tanto main siguió hasta D. Las dos líneas existen a la vez y no se molestan.

Lo que hay que retener

Las flechas apuntan hacia atrás. Un commit sabe de dónde viene, pero no sabe qué vino después. Por eso podés crear ramas nuevas desde cualquier punto sin tocar nada de lo anterior.

Una rama es solo un puntero

Esta es la idea que hace que todo lo demás se entienda

Una rama no es una copia de los archivos. Es una etiqueta móvil que apunta a un commit. Crear una rama es crear un post-it: por eso es instantáneo, aunque el proyecto pese 2 GB.

1 Crear una rama es instantáneo

No se copia nada. Se escribe un archivo de 41 bytes con un hash adentro.

2 Cuando hacés commit, el puntero avanza

La rama en la que estás parado se mueve sola al commit nuevo.

3 Borrar una rama no borra commits

Se borra la etiqueta. Los commits siguen ahí hasta que Git los recolecte.

4 HEAD apunta a la rama actual

Es el puntero al puntero: 'dónde estoy parado ahora'.

Los comandos que vas a usar todos los días

Cada uno mapea a una operación sobre el grafo

git status

¿Qué cambió y dónde estoy parado?

Corrélo antes y después de todo

git add <archivo>

Poner un cambio en el staging

git add . agrega todo lo modificado

git commit -m "..."

Crear un commit nuevo

El puntero de la rama avanza

git log --oneline --graph

Ver el grafo del historial

El mejor comando para entender dónde estás

git branch <nombre>

Crear una etiqueta nueva acá

No te cambia de rama

git checkout <rama>

Moverte a otra rama

Los archivos cambian en tu disco

git checkout -b <rama>

Crear y moverte, en un paso

El atajo que se usa en la práctica

git merge <rama>

Traer otra rama a la actual

Puede generar conflictos

git push / git pull

Sincronizar con GitHub

push sube, pull baja

02

BLOQUE 2 · 30 MINUTOS

Ramas, merge y conflictos

Cómo se separan dos líneas de trabajo, cómo se vuelven a unir y qué pasa cuando chocan.

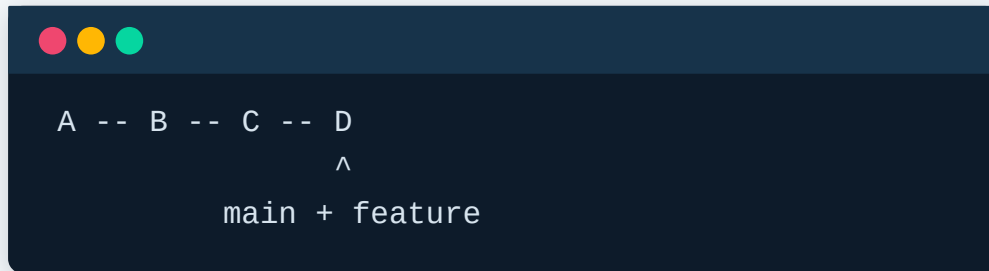
Dos formas de fusionar

Git elige una u otra según lo que pasó en main mientras trabajabas

FAST-FORWARD

main no avanzó mientras trabajabas

Git no necesita fusionar nada: simplemente mueve el puntero de main hacia adelante, hasta el último commit de tu rama.

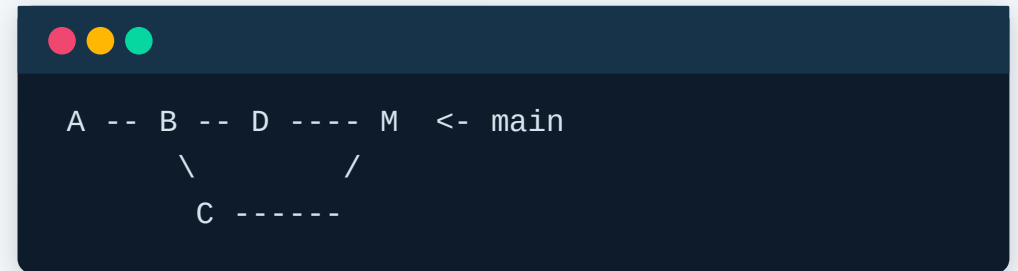


El historial queda en línea recta.

MERGE COMMIT

main avanzó por otro lado

Las dos líneas divergieron. Git crea un commit nuevo con DOS padres, que une las dos historias.



Acá es donde pueden aparecer conflictos.

El modelo del ejercicio evaluado

La plataforma usa un modelo simplificado. Estas son sus cuatro reglas exactas.

```
clone <repo>
```

Arranca en la rama main con 0 commits

```
commit
```

Suma 1 commit a la rama actual

```
branch <nombre>
```

Crea una rama con el MISMO conteo que la rama actual

```
checkout <nombre>
```

Cambia de rama. Si la rama no existe, se ignora el comando

```
merge <nombre>
```

Suma el conteo de la otra rama al de la rama actual

Ojo: este modelo NO es Git real

En Git de verdad, un merge no duplica los commits que las dos ramas ya compartían. El ejercicio simplifica a propósito, para que el foco esté en seguir el estado de cada rama.

Traza paso a paso

El caso básico del ejercicio, comando por comando

#	COMANDO	main	feature	RAMA ACTUAL
1	clone repo1	0	—	main
2	commit	1	—	main
3	branch feature	1	1	main
4	checkout feature	1	1	feature
5	commit	1	2	feature
6	checkout main	1	2	main
7	merge feature	3	2	main

Salida esperada: main 3

Conflictos de merge

Ocurren cuando dos ramas tocaron LA MISMA LÍNEA del mismo archivo

CUÁNDO HAY CONFLICTO

- Las dos ramas editaron la misma línea
- Una borró un archivo que la otra modificó

CUÁNDO NO

- Editaron archivos distintos
- Editaron el mismo archivo en líneas distintas

así se ve un conflicto en el archivo

```
<<<<<< HEAD
<h1>Hola DevOps</h1>
=====
<h1>Bienvenidos a DevOps</h1>
>>>>>> feature/titulo
```

Cómo leerlo

- Arriba del === lo que tenías (HEAD)
- Abajo del === lo que traés
- Editás el archivo, borrás los marcadores y dejás la versión correcta

Resolver un conflicto, paso a paso

Cuatro pasos. No hay magia y no hace falta borrar la carpeta.

1

Leé qué archivos están en conflicto

git status te los lista bajo "Unmerged paths"

2

Abrí el archivo y decidí

Quedate con una versión, con la otra, o escribí una tercera que combine ambas

3

Borrá los marcadores

Las líneas con <<<<<<, ===== y >>>>>> no van en el resultado final

4

Marcá como resuelto y cerrá el merge

git add <archivo> y después git commit

● ● ● el botón de pánico

```
git merge --abort    # si te arrepentiste: vuelve todo al estado anterior al merge
```

ACTIVIDAD 1 · 10 MINUTOS

¿En qué estado queda el repo?

Aplicá las cinco reglas y respondé por el chat con el formato: rama_actual conteo

● ● ● secuencia A

```
clone repo1
commit
branch feature1
branch feature2
checkout feature1
commit
checkout feature2
commit
commit
checkout main
merge feature1
merge feature2
```

● ● ● secuencia B · secuencia C

```
clone repo1
branch dev
checkout dev
checkout main
merge dev
commit

clone repo1
commit
checkout unknown
commit
```

Pista: escribí una columna por rama y andá anotando. No lo hagas de memoria.

Break

10 minutos

Volvemos con flujos de trabajo y GitHub Actions

03

BLOQUE 3 · 20 MINUTOS

Flujos de trabajo

Git te da las piezas. El flujo de trabajo es el acuerdo del equipo sobre cómo usarlas.

Tres flujos de trabajo

De más simple a más ceremonioso. El más simple suele ser el correcto.

GitHub Flow

Una rama main + ramas de feature de vida corta. Se integra por pull request y se despliega desde main.

Cuándo: La mayoría de los equipos web y SaaS

Recomendado para empezar

Trunk-based

Todos commitean a main varias veces al día. Lo no terminado se esconde detrás de feature flags.

Cuándo: Equipos maduros con CI muy sólida

Máxima velocidad, máxima exigencia

Git Flow

Ramas develop, release, hotfix y feature. Muchas reglas, muchos merges.

Cuándo: Software con versiones formales (desktop, embebido)

Suele ser demasiado para una app web

En este curso usamos GitHub Flow: es el que mejor se lleva con CI/CD y el que van a encontrar en la mayoría de los equipos.

El pull request

El lugar donde el código se discute antes de entrar a main

QUÉ ES

Una propuesta de cambio: "quiero traer mi rama a main, ¿lo revisan?". Abre una conversación sobre un conjunto de commits, con el diff a la vista.

PARA QUÉ SIRVE

- Segunda mirada: se detectan errores antes de producción
- Sharing: el conocimiento circula por el equipo
- Es el gancho natural para la automatización



El paso 4 es donde se conectan Git y CI: abrir un pull request dispara el pipeline automáticamente. Nadie aprueba nada hasta que el pipeline esté en verde.

Escribir buenos commits

El mensaje lo vas a leer vos mismo dentro de seis meses, sin acordarte de nada

MAL

```
"cambios"  
"fix"  
"asdasd"  
"ahora si"  
"cambios varios del viernes"
```

BIEN — CONVENTIONAL COMMITS

```
feat: agrego login con Google  
fix: corrijo cálculo del total  
docs: actualizo el README  
refactor: simplifico el parser  
test: sumo casos de borde
```

Un commit, un cambio

Si el mensaje necesita un "y", probablemente sean dos commits

En presente e imperativo

"agrego X", no "agregué X": describe qué hace el commit al aplicarse

El prefijo da contexto

feat, fix, docs, refactor, test, chore. Permite generar changelogs automáticos

04

BLOQUE 4 · 25 MINUTOS

GitHub Actions

El nudo del bucle: verificar cada cambio automáticamente, sin que nadie apriete un botón.

Qué es GitHub Actions

Un servidor que ejecuta comandos por vos cuando pasa algo en el repositorio

Workflow

El archivo YAML completo. Vive en `.github/workflows/`

Evento (on)

Qué lo dispara: un push, un pull request, un horario, un botón

Job

Un conjunto de pasos que corren en la misma máquina. Varios jobs corren en paralelo

Runner

La máquina virtual donde corre el job. GitHub te da una limpia cada vez

Step

Un paso: ejecutar un comando o usar una acción hecha por otro

Action

Un bloque reutilizable, ej. `actions/checkout` para traer tu código

Es gratis para repositorios públicos. No necesitás servidor propio ni instalar nada.

Tu primer workflow, línea por línea

Este archivo va en `.github/workflows/ci.yml`

```
.github/workflows/ci.yml

name: CI

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  verificar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Validar HTML
        run: test -f docker/index.html
      - name: Construir imagen
        run: docker build -t app:ci docker/
```

name	Cómo se ve el workflow en la pestaña Actions
on	Los eventos que lo disparan: push a main o cualquier PR
jobs	La lista de trabajos. Acá hay uno solo: "verificar"
runs-on	El sistema operativo del runner. ubuntu-latest es lo normal
uses	Reutiliza una acción de otro. checkout trae tu código al runner
run	Ejecuta un comando de shell, igual que en tu terminal

Qué pasa cuando hacés push

El ciclo completo, en menos de un minuto

- 1 Hacés git push** El evento llega a GitHub
- 2 GitHub lee .github/workflows/** Encuentra el workflow y ve que el evento coincide
- 3 Levanta un runner limpio** Una máquina virtual Ubuntu nueva, sin nada tuyo adentro
- 4 Ejecuta los steps en orden** checkout, validaciones, build. Si uno falla, se corta ahí
- 5 Reporta el resultado** Tilde verde o cruz roja en tu commit y en el pull request

Eso es Integración Continua funcionando. Nadie apretó nada: el push fue el disparador.

DEMO EN VIVO · 12 MINUTOS

Del conflicto al pipeline en verde

Seguime en paralelo con el repo de la práctica de la clase pasada.

conflicto y resolución

```
# 1. Provocar un conflicto
git checkout -b rama-a
# editar la línea 1
git commit -am "feat: version A"

git checkout main
# editar LA MISMA línea
git commit -am "feat: version B"
git merge rama-a    # ← CONFLICTO
```

github actions

```
# 2. Primer pipeline
mkdir -p .github/workflows
# crear ci.yml
git add .github/
git commit -m "ci: primer workflow"
git push

# → pestaña Actions en GitHub
```

Cerrá rompiendo el pipeline a propósito: borrá un archivo que el workflow verifica y mostrará la cruz roja.

Actividad 2: chequeo rápido

Verdadero o falso. Respondé por el chat, después las revisamos.

- 1 Crear una rama copia todos los archivos del proyecto. V / F
- 2 Un commit guarda solo las líneas que cambiaron respecto del anterior. V / F
- 3 Si borrás una rama, se borran los commits que tenía. V / F
- 4 Un conflicto ocurre cuando dos ramas modifican la misma línea del mismo archivo. V / F
- 5 GitHub Actions necesita que instales un servidor propio para funcionar. V / F
- 6 Abrir un pull request puede disparar un pipeline automáticamente. V / F

Las respuestas están en el guión docente. Cada una es una excusa para repasar, no para evaluar.

Para la próxima clase

Dos consignas: la evaluada de la plataforma y la práctica del repo

1 Ejercicio de la plataforma

- Simulador de Git en Python: lee N comandos y devuelve rama y conteo
- Aplicá las cinco reglas de la slide 14, no la semántica real de Git
- Probá con los 4 casos visibles antes de enviar

2 Práctica sobre tu repo

- Provocá y resolvé un conflicto de merge, documentándolo
- Trabajá con una rama y abrí un pull request real
- Agregá el workflow de CI y dejalo en verde

Próxima clase — Módulo 3: Domina la containerización con Docker

Capas, volúmenes, redes y multi-stage builds para imágenes livianas.

Lo que nos llevamos

- 1 Un commit es una foto completa con un puntero a su padre; el historial es un grafo
- 2 Una rama es solo una etiqueta móvil: por eso crearla es instantáneo y borrarla no pierde nada
- 3 Un merge es fast-forward si main no avanzó, y un commit con dos padres si divergieron
- 4 Un conflicto no es un error: es Git pidiendo una decisión que no puede tomar solo
- 5 GitHub Actions convierte un push en un pipeline: ahí empieza la Integración Continua

¿Preguntas?